

Traveling Salesman Problem by Branch and Bound

1 Problem Definition

Traveling Salesman Problem

Given n cities and the distance between every pair of cities, find a shortest tour that starts from one city, visits every other city exactly once, and returns to the starting city.

A tour is also called a Hamiltonian circuit. For cities

$$\{a, b, c, d, e\},$$

one possible tour is

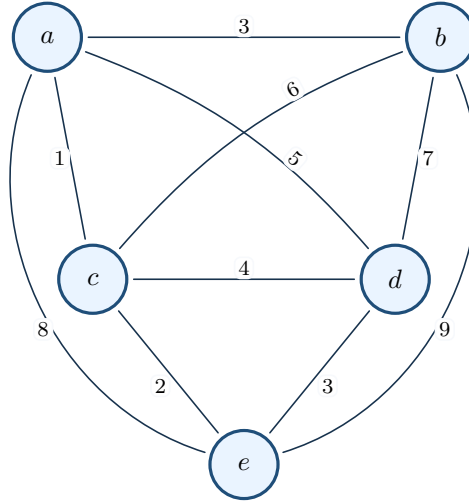
$$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a.$$

The objective is to minimize the total length of the tour:

$$\text{tour length} = \sum \text{lengths of the selected tour edges.}$$

2 Example Graph

The example uses the following undirected weighted graph.



3 Why Branch and Bound Is Useful

If the starting city is fixed as a , then a brute-force search checks

$$(n - 1)!$$

possible orders of the remaining cities. For 5 cities, this is

$$4! = 24$$

tours.

Branch and Bound avoids checking every tour by removing branches that cannot beat the best tour already found.

Main Idea

Build tours step by step. For each partial tour, compute a lower bound. If the lower bound is already not better than the best complete tour found so far, prune that branch.

4 Lower Bound for TSP

In any Hamiltonian circuit, every city has degree 2, because one edge enters the city and one edge leaves it.

Therefore, a simple lower bound is obtained by taking the two smallest incident edge lengths for every city.

TSP Lower Bound

For each city i , let s_i be the sum of the two smallest edge lengths incident with city i . Then

$$LB = \left\lceil \frac{\sum_{i=1}^n s_i}{2} \right\rceil.$$

We divide by 2 because every edge is counted at both of its endpoints.

5 Root Lower Bound

Find the two smallest incident edge lengths for each city.

City	Two smallest incident edges	Sum
a	$ac = 1, ab = 3$	$1 + 3 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

So,

$$\sum s_i = 4 + 9 + 3 + 7 + 5 = 28.$$

Therefore,

$$LB_{\text{root}} = \left\lceil \frac{28}{2} \right\rceil = 14.$$

6 Bound When Some Edges Are Forced

Sometimes a branch already forces some edges to be included in the tour. Then the bound must respect those required edges.

For example, suppose the partial tour begins with

$$a \rightarrow d.$$

Then the edge ad must be included at both city a and city d .

$$\begin{aligned} LB &= \left\lceil \frac{(1+5) + (3+6) + (1+2) + (3+5) + (2+3)}{2} \right\rceil \\ &= \left\lceil \frac{6+9+3+8+5}{2} \right\rceil \\ &= \left\lceil \frac{31}{2} \right\rceil \\ &= 16. \end{aligned}$$

Meaning of the Bound

A lower bound is not always the actual tour length. It is an optimistic estimate. If even this optimistic estimate is too large, the branch cannot contain the optimal tour.

7 Branching Strategy

To avoid generating the same undirected tour many times, fix the starting city as a . Then branch by choosing the next city after a .

The first-level branches are:

$$a, b \quad a, c \quad a, d \quad a, e.$$

For each branch, compute a lower bound and expand the most promising live node.

Node	Meaning	Bound or status
a, b	tour begins $a \rightarrow b$	14
a, c	tour begins $a \rightarrow c$	14, skipped only by symmetry rule
a, d	tour begins $a \rightarrow d$	16
a, e	tour begins $a \rightarrow e$	19

How the Table Bounds Are Calculated

For each branch, the lower bound uses the same rule:

$$LB = \left\lceil \frac{\sum s_i}{2} \right\rceil,$$

where s_i is the sum of two required or cheapest incident edges for city i . If a branch forces an edge, that forced edge must be included in the two edges for its endpoint cities.

Branch a, b :

The branch starts with edge $ab = 3$. This edge is already one of the two cheapest choices for both a and b , so the root bound stays the same.

City	two edges used in bound	s_i
a	$ac = 1, ab = 3$	$1 + 3 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB = \left\lceil \frac{4 + 9 + 3 + 7 + 5}{2} \right\rceil = \left\lceil \frac{28}{2} \right\rceil = 14.$$

Branch a, c , if it is not skipped:

The branch starts with edge $ac = 1$. This branch is valid. If we decide not to use symmetry skipping, then its lower bound is calculated exactly like the other branches. Since ac is already one of the two cheapest incident edges for both a and c , the lower bound again remains the same as the root bound.

City	two edges used in bound	s_i
a	$ac = 1, ab = 3$	$1 + 3 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB = \left\lceil \frac{4 + 9 + 3 + 7 + 5}{2} \right\rceil = \left\lceil \frac{28}{2} \right\rceil = 14.$$

Safe Interpretation

The branch a, c is not removed by the lower-bound test. If no symmetry-breaking rule is stated, a, c should be treated as a normal live branch with lower bound 14.

Branch a, d :

The branch forces edge $ad = 5$. Therefore, ad must be counted for city a and city d .

City	two edges used in bound	s_i
a	$ac = 1, ad = 5$	$1 + 5 = 6$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, da = 5$	$3 + 5 = 8$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB = \left\lceil \frac{6 + 9 + 3 + 8 + 5}{2} \right\rceil = \left\lceil \frac{31}{2} \right\rceil = 16.$$

Branch a, e :

The branch forces edge $ae = 8$. Therefore, ae must be counted for city a and city e .

City	two edges used in bound	s_i
a	$ac = 1, ae = 8$	$1 + 8 = 9$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ea = 8$	$2 + 8 = 10$

$$LB = \left\lceil \frac{9 + 9 + 3 + 7 + 10}{2} \right\rceil = \left\lceil \frac{38}{2} \right\rceil = 19.$$

Why the Branch a, c Is Not Used

The branch a, c is not invalid. It is skipped only to avoid duplicate tours.

In an undirected TSP, a tour and its reverse are the same cycle. For example,

$$a \rightarrow c \rightarrow e \rightarrow d \rightarrow b \rightarrow a$$

has the same edges as the reversed tour

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a.$$

So a tour that begins with $a \rightarrow c$ can often be represented in reverse as a tour that begins with $a \rightarrow b$. To avoid generating both versions, the search fixes a direction/order and skips a, c at the first level.

Key Point

a, c is valid, but unnecessary in this organized search.

8 Completing Candidate Tours

After expanding the promising branch a, b , the tour must still visit the remaining cities

$$\{c, d, e\}$$

and then return to the starting city a .

So every candidate in this part has the form

$$a \rightarrow b \rightarrow \text{remaining cities in some order} \rightarrow a.$$

For example,

$$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$$

means the selected edges are

$$ab, \quad bc, \quad ce, \quad ed, \quad da.$$

The length is computed by adding the edge weights from the graph.

Step-by-Step Search Under a, b

After $a \rightarrow b$, the next city can be c, d , or e . The search compares these live branches by their lower bounds.

Partial path	Meaning	Bound/status
a, b, c	continue through city c	$LB = 16$
a, b, d	continue through city d	$LB = 16$
a, b, e	continue through city e	$LB = 19$

How These Bounds Are Calculated

For a partial path, all edges already in the path are forced. The lower-bound calculation must include those forced edges at their endpoint cities. Each city still needs two incident edges in the final tour.

Path a, b, c :

The forced edges are

$$ab = 3, \quad bc = 6.$$

City	two edges used in bound	s_i
a	$ab = 3, ac = 1$	$3 + 1 = 4$
b	$ba = 3, bc = 6$	$3 + 6 = 9$
c	$cb = 6, ca = 1$	$6 + 1 = 7$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB(a, b, c) = \left\lceil \frac{4 + 9 + 7 + 7 + 5}{2} \right\rceil = \left\lceil \frac{32}{2} \right\rceil = 16.$$

Path a, b, d :

The forced edges are

$$ab = 3, \quad bd = 7.$$

City	two edges used in bound	s_i
a	$ab = 3, ac = 1$	$3 + 1 = 4$
b	$ba = 3, bd = 7$	$3 + 7 = 10$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$db = 7, de = 3$	$7 + 3 = 10$
e	$ec = 2, ed = 3$	$2 + 3 = 5$

$$LB(a, b, d) = \left\lceil \frac{4 + 10 + 3 + 10 + 5}{2} \right\rceil = \left\lceil \frac{32}{2} \right\rceil = 16.$$

Path a, b, e :

The forced edges are

$$ab = 3, \quad be = 9.$$

City	two edges used in bound	s_i
a	$ab = 3, ac = 1$	$3 + 1 = 4$
b	$ba = 3, be = 9$	$3 + 9 = 12$
c	$ca = 1, ce = 2$	$1 + 2 = 3$
d	$de = 3, dc = 4$	$3 + 4 = 7$
e	$eb = 9, ec = 2$	$9 + 2 = 11$

$$LB(a, b, e) = \left\lceil \frac{4 + 12 + 3 + 7 + 11}{2} \right\rceil = \left\lceil \frac{37}{2} \right\rceil = 19.$$

The branches a, b, c and a, b, d are more promising than a, b, e , because

$$16 < 19.$$

If live branches have different lower bounds, choose the branch with the smallest lower bound. For example, if

$$LB(a, b, c) = 16, \quad LB(a, b, d) = 17, \quad LB(a, b, e) = 18,$$

then the next branch to expand would be

$$\boxed{a, b, c}$$

because

$$16 < 17 < 18.$$

If two branches have the same lower bound, either one may be expanded first. A common rule is to choose the branch that was generated first.

So in this example, a, b, c and a, b, d are tied:

$$LB(a, b, c) = LB(a, b, d) = 16.$$

The algorithm may expand either one first, but it should keep the rule consistent.

Later, when a complete tour of length 16 is found, the branch a, b, e is no longer worth expanding because

$$LB(a, b, e) = 19 \geq 16 = \text{bestTourLength}.$$

Why $a \rightarrow b \rightarrow e$ Is Not Continued

$a \rightarrow b \rightarrow e$ is valid, but its lower bound 19 cannot beat the best tour length 16.

Next Step: Expand a, b, c and a, b, d

After a, b, e is set aside, the useful live branches are

$$a, b, c \quad \text{and} \quad a, b, d.$$

When to Recalculate LB

If many cities are still unvisited, compute a new lower bound for the partial path and compare it with the current best tour length.

If only one or two cities are left, it is usually clearer to finish the possible tours directly and calculate their actual lengths instead of estimating another bound.

Here, after a, b, c or a, b, d , only two cities remain. Therefore, we complete the tours directly and compute their actual lengths.

Expand a, b, c :

The path already uses

$$a \rightarrow b \rightarrow c.$$

The remaining cities are

$$\{d, e\}.$$

There are only two possible completions:

Choice after a, b, c	Complete tour forced by the choice	Length
Go to d next	$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$	$3 + 6 + 4 + 3 + 8 = 24$
Go to e next	$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$	$3 + 6 + 2 + 3 + 5 = 19$

So under a, b, c , the better completion is

$$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$$

with length

$$19.$$

Expand a, b, d :

The path already uses

$$a \rightarrow b \rightarrow d.$$

The remaining cities are

$$\{c, e\}.$$

Again, there are only two possible completions:

Choice after a, b, d	Complete tour forced by the choice	Length
Go to c next	$a \rightarrow b \rightarrow d \rightarrow c \rightarrow e \rightarrow a$	$3 + 7 + 4 + 2 + 8 = 24$
Go to e next	$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$	$3 + 7 + 3 + 2 + 1 = 16$

So under a, b, d , the better completion is

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$$

with length

$$16.$$

Next-Step Conclusion

After expanding a, b, c and a, b, d , the best tour found has length 16.

Tour	Length	Status
$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a$	$3 + 6 + 4 + 3 + 8 = 24$	first complete tour
$a \rightarrow b \rightarrow c \rightarrow e \rightarrow d \rightarrow a$	$3 + 6 + 2 + 3 + 5 = 19$	better tour
$a \rightarrow b \rightarrow d \rightarrow c \rightarrow e \rightarrow a$	$3 + 7 + 4 + 2 + 8 = 24$	inferior tour
$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$	$3 + 7 + 3 + 2 + 1 = 16$	best tour

Length Calculations

Tour	Edge-by-edge calculation	Length
a, b, c, d, e, a	$ab + bc + cd + de + ea = 3 + 6 + 4 + 3 + 8$	24
a, b, c, e, d, a	$ab + bc + ce + ed + da = 3 + 6 + 2 + 3 + 5$	19
a, b, d, c, e, a	$ab + bd + dc + ce + ea = 3 + 7 + 4 + 2 + 8$	24
a, b, d, e, c, a	$ab + bd + de + ec + ca = 3 + 7 + 3 + 2 + 1$	16

Updating the Best Tour

When the first complete tour is found,

$$a \rightarrow b \rightarrow c \rightarrow d \rightarrow e \rightarrow a,$$

its length is 24, so initially

$$\text{bestTourLength} = 24.$$

Then the next complete tour has length 19, so it improves the current best value:

$$19 < 24.$$

Finally,

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$$

has length 16, and

$$16 < 19.$$

Therefore, the current best tour is updated to length 16.

The best complete tour found is

$$a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$$

with length

$$16.$$

9 Pruning

For minimization problems, the pruning rule is:

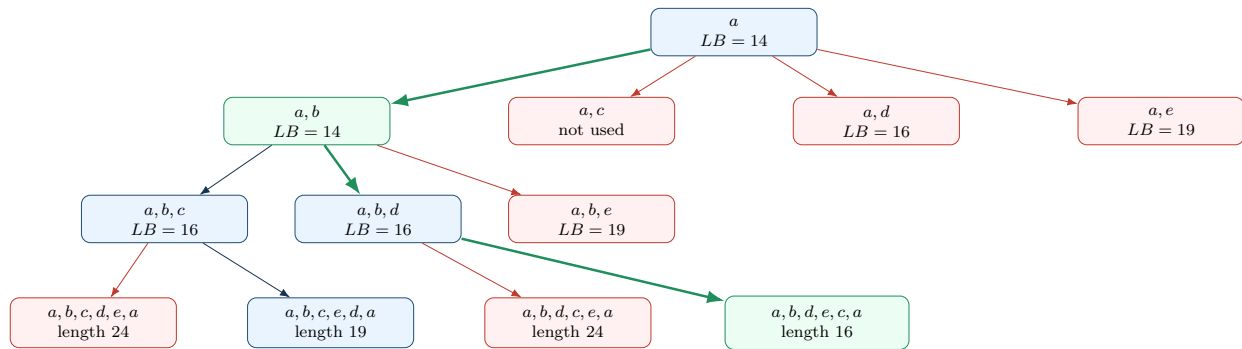
Pruning Rule

$$LB \geq \text{bestTourLength} \implies \text{prune the node.}$$

Once a tour of length 16 is found, every live branch with lower bound at least 16 cannot improve the answer. Such branches can be terminated.

Node	Bound or length	Decision
a, d	$LB = 16$	prune
a, e	$LB = 19$	prune
a, b, e	$LB = 19$	prune
a, b, d, e, c, a	length 16	incumbent optimal tour

10 State-Space Tree



11 Pseudocode

TSP Branch and Bound

```
TSPBranchAndBound(graph):
    fix the starting city as a
    bestTourLength = infinity
    bestTour = none

    create the root node with path [a]
    compute root.lowerBound
    liveNodes = {root}

    while liveNodes is not empty:
        node = remove the live node with smallest lowerBound

        if node.lowerBound >= bestTourLength:
            continue

        if node represents a complete tour:
            if node.length < bestTourLength:
                bestTourLength = node.length
                bestTour = node.tour
            continue

        create children by adding one allowed unvisited city

        for each child:
            compute child.lowerBound
            if child.lowerBound < bestTourLength:
                add child to liveNodes

    return bestTour and bestTourLength
```

12 Correctness Intuition

Branch and Bound does not ignore a branch randomly. A branch is pruned only when its lower bound proves that no completion of that partial tour can be better than the best complete tour

already found.

Therefore, if a branch could contain a better tour, its lower bound would be smaller than the current best tour length, and the algorithm would keep it live.

13 Complexity

Even with pruning, the worst case may still require checking many permutations of the cities. With one starting city fixed, the number of possible tours is proportional to

$$(n - 1)!.$$

For an undirected symmetric TSP, reverse tours are the same, so the number of distinct tours is

$$\frac{(n - 1)!}{2}.$$

Worst-case time is factorial.

Branch and Bound is useful because good bounds can greatly reduce the number of tours explored in practice.

14 Final Review

TSP Branch and Bound Essentials

TSP asks for a shortest Hamiltonian circuit.

$$LB = \left\lceil \frac{\sum s_i}{2} \right\rceil, \quad s_i = \text{sum of two smallest incident edges at city } i.$$

For minimization, prune when $LB \geq \text{bestTourLength}$.

Best tour here: $a \rightarrow b \rightarrow d \rightarrow e \rightarrow c \rightarrow a$.

Minimum tour length = 16.